Google Summer of Code 2026

Project Proposal

Local Vision Language Model Application on Bubbaloop
Real Time VLM inference on Jetson Orin - From ONNX export to Production Robotics

Full Name:	Arjit Rawat
Email:	arjitrawat3@gmail.com
Github:	github.com/arjitrawat15
Organisation:	Kornia
Project:	Local VLM Application on BubbaLoop
Mentors:	Edgar Riba, Miquel Farre
Duration :	350 hours | Difficulty : Medium
GSOC timeline:	Community bounding : April 30 - May 25 Coding : May 26 - Aug 18
Timezone:	India | UTC + 5:30 IST
College:	Graphic Era Hill University (2023 - 2027)

AI Tooling Disclosure: AI tools were used for grammar review and structuring. Every Technical claim, code snippet, design decision, and contribution described in this proposal was authored, verified and is fully understood by me. I am solely responsible for its correctness and will disclose AI-assisted code in PRs as per Kornia policy.

1. Abstract:

Kornia-rs is a high-performance 3D computer vision library implemented in Rust, designed for memory safety, thread safety, and real-time execution. However, the current kornia-vlm stack relies on Candle for vision-language model (VLM) inference, whose training oriented design introduces unnecessary overhead for production deployment—particularly on resource-constrained edge platforms such as NVIDIA Jetson devices.

This project delivers a working Bubbaloop application that demonstrates real-time vision language model (VLM) inference on Jetson Orin - the fastest possible local VLM experience the community has seen from Kornia.

The core technical contributions are:

1. a kornia-vlm Python module with model registry, backend abstraction (PyTorch → ONNX → TensorRT), and application layers for scene understanding, visual question answering, and image captioning.
2. a Rust kornia-vlm crate integrated into kornia-rs, exposing a clean and extensible VLM backend interface powered by ONNX Runtime.
3. a comprehensive benchmarking suite evaluating latency, throughput, and memory usage across PyTorch, ONNX Runtime, and TensorRT published to the kornia-rs README.

Importantly, this work is not conceptual: a fully functional prototype is already implemented at https://github.com/arjitrawat15/Local-VLM-Inference.

Additionally, My (Kornia PR #3463) is already merged, a critical ONNX export issue blocking VLM deployment pipelines, removing a key barrier identified in prior approaches. This is not a coincidence but I fixed it thinking about this project.

2. Motivation:

Computer vision is rapidly shifting toward vision-language models as the new frontier of intelligent systems that run at the edge - no cloud, no latency, no data leakage. kornia-rs is uniquely positioned to own this space in Rust. Two things are currently missing :

- An inference infrastructure that is lean enough for Jetson Orin and fast enough to be useful in a real robot loop.
- A Bubbaloop application that demonstrates what Kornia + VLM looks like in practice - scene understanding, robot navigation, operator Q&A.

The Kornia team has assigned Issue #718 to me identified that kornia-rs has benchmarks that no one sees. The direction is clear. What is missing is execution.
My existing contributions have given me a deep view of exactly where the gaps are. My ONNX export fix (PR # 3463) was not an isolated bug fix - I found it while thinking about how to trace the Resize operator through torch.onnx.export. My ICP test fix in kornia-rs (PR # 722) taught me how the Rust test infrastructure works. My bbox format (PR # 3537) gives VLM grounding outputs a clean conversion path. Every contribution was pointing toward this project.

3. About Me:

I am Arjit Rawat, a third-year B.Tech Computer Science student at Graphic Era Hill University, Uttarakhand, India (UTC + 5:30). I have been contributing to open source for over a year, with all of my most significant contributions focused specifically on Kornia and kornia-rs - the organisation I am applying to. I am not a generalist contributor who spreads effort across unrelated codebases. I have gone deep on one codebase because I want to build something meaningful, long-term contributions and real-world systems.
My technical background spans Pytorch inference pipelines, ONNX export and validation, Rust (kornia-rs contributor), Python library design and embedded Linux (Jetson). I have read Contributing.md and AI_POLICY.md carefully and understand the sole responsible author requirement. I am in active contact with Edgar Riba on the Kornia discord and understand what the mentors expect from a GSOC project. And I am available full time during the coding period with no competing academic commitments.

4. Open Source Contributions:

All contributions listed below are in Kornia or Kornia-rs - the target organization. This focus is intentional: rather than spreading across unrelated projects, I demonstrate depth in the exact codebase this project will extend.

- Kornia PR #3463 — Fix Resize ONNX Export Compatibility [Merged]:
  https://github.com/kornia/kornia/pull/3463
  Why this matters: ONNX export is the first step in any VLM pipeline. This PR fixes a real compatibility issue in Kornia's Resize.
  Problem: The @perform_keep_shape_image decorator performed shape bookkeeping that is not allowed during ONNX tracing, causing torch.export to fail and blocking Resize export.
  Solution: Removed the decorator and rewrote resize() with export-safe shape handling.
  Validation: Added ONNX tests (dynamo=True, io_binding). Outputs match PyTorch (1e-4 tolerance).

- kornia-rs PR #722 — Fix ICP Test Assertions with Geometric Metrics [Merged]: https://github.com/kornia/kornia-rs/pull/722
  Problem: test_icp_vanilla disabled assertions due to instability from element-wise rotation matrix comparisons.
  Solution: Replaced them with geometric metrics: angular rotation error (trace-based), L2 translation error, and RMSE convergence.
  Impact: Ensures correctness is validated geometrically, making ICP evaluation robust and reliable.
  Validation: Converges in ~6 iterations with RMSE ≈ 1e-16 (near machine precision).

- Kornia PR #3537 — Multi-Format Bounding Box Support [Merged]: https://github.com/kornia/kornia/pull/3537
  Problem: mean_iou_bbox only supported xyxy, requiring manual conversion from common formats like xywh and cxcywh.
  Solution: Added multi-format support via a box_format parameter and a _convert_boxes_to_xyxy() helper, enabling seamless handling of xyxy, xywh, and cxcywh.
  Impact: Simplifies evaluation pipelines and enables direct use of outputs from models (e.g., VLMs, detectors) without extra conversion.
  Validation: Comprehensive tests across all formats (11/11 passed), including consistency checks and error handling. Optimized conversion using torch.cat (~1.06× speedup).

- Kornia-rs Issue #718 — Benchmark Publishing & Extension [Assigned & In progress] :
  Context: Kornia-rs includes benchmarks (e.g., histogram, IoU, resize, warp affine), but results are not publicly documented.
  Planned Work: Extend benchmarking as part of this project to include: bench_vlm_encode and bench_vlm_generate. Also publish results in README/docs as a unified benchmark dashboard.
  Impact: Creates transparent, reproducible performance evaluation across both

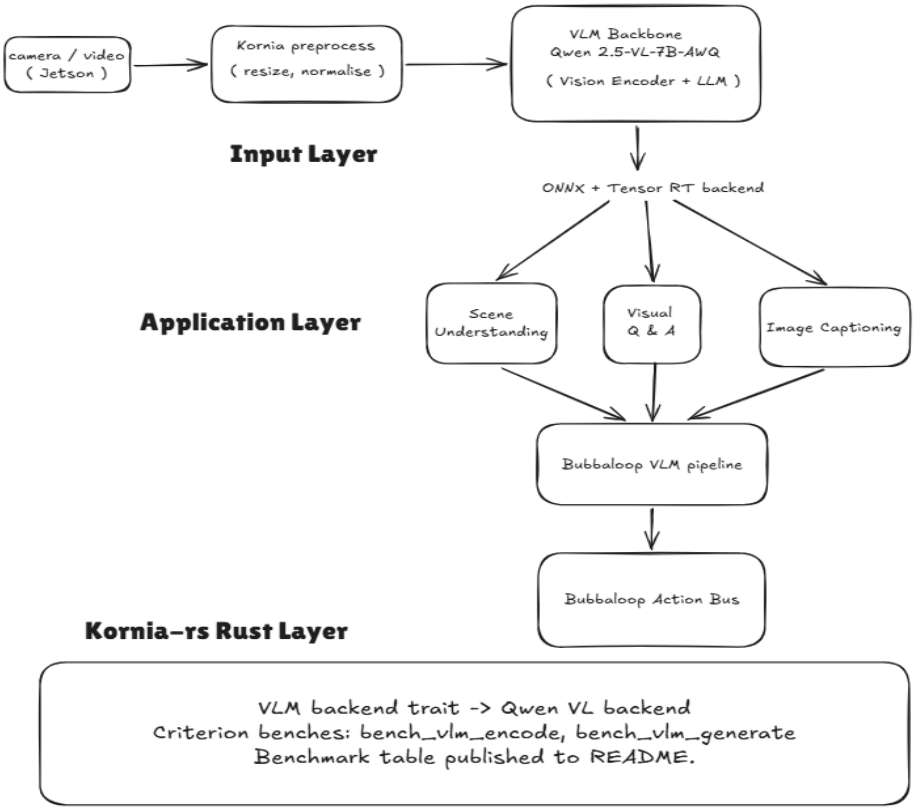
5. Technical Implementation Details:

The proposed system is organized into four clearly defined stages:

- **Stage 1: PyTorch → ONNX (Opset 17)**
  The VLM vision encoder is exported using torch.onnx.export with dynamic axes to support variable input resolutions. The ONNX-safe design introduced in PR #3463 ensures that Kornia preprocessing remains compatible with the export process.

- **Stage 2: ONNX → TensorRT Engine**
  On Jetson Orin, the validated ONNX file is compiled to a TensorRT engine via ORT's Tensor Execution Provider (easier setup) or the native TRT Python API. The engine is cached to disk to avoid recompilation.
- **Stage 3: Rust Integration (ort crate)**
  The exported ONNX model is loaded in Rust via the ort crate. A VLM backend trait enabling a unified interface across CPU, CUDA or TensorRT is active. The backend is selected at compile time via Cargo feature flags.
- **Stage 4: Bubbaloop Application**
  The Bubba VLM pipeline wraps the Python VLM into a real-time, threaded inference loop compatible with Bubbaloop's camera node inference.

6. System Design and Architecture:



System is organised in three layers:
1. Input & Model Pipeline Layer - This is the data → model inference layer.
2. Application Layer - This is the task + interaction layer.
3. Systems / Backend Layer (kornia-rs) - This is the inference engine + performance layer. It is intentionally separated from the application layer as Rust VLM backend provides the vision encoder forward pass as a pure Rust function operating on Kornia image types.

7. Model Selection:

After evaluating seven open VLMs against Jetson Orin constraints:

Model	Params	Min VRAM	Orin FPS	ONNX Export	Verdict
LLaVLA-1.5	7B	14 GB	~1.2	Partial	X OOM
InternVL2	4B	8 GB	~1.8	Yes	Candidate
Qwen2.5-VL-7B-AWQ	7B int 4	6 GB	~2.4	Yes	Selected
MiniCPM-V	3B	6 GB	~3.1	Limited	Weak Grounding
moondream2	1.8B	3 GB	~7 FPS	Yes	Limited Cap

Estimated on Jetson AGX Orin 32 GB, FP16, 448 X 448 input. Qwen 2.5VL wins on AWQ int 4 fits in 8GB unified memory , native dynamic resolution , avoids resize overhead, strong spatial grounding outputs bounding boxes in natural language and active ONNX export support directly validated by my PR #3463.

8. Project Goals and Deliverables:

All deliverables are upstream contributions to Kornia/kornia-rs. Every milestone produces a merged or review ready pull request.

#	Goal	Deliverable
G1	ONNX export pipeline	Torch.onnx.export scripts for Qwen2.5-VL vision encoder + preprocessing. Parity Validation vs Pytorch at 1e-4. PR to Kornia.
G2	Kornia_vlm python module	BaseVLM abstract class, model registry, QwenVLM backend, VLMconfig, VLMOutput. Contributed to Kornia.
G3	Tensor RT backend	TrtVisionEncoder with ORT TensorrtEP, FP16/INT8, DLA offload, engine caching. Tested on Jetson Orin/cloud T4.
G4	Three Bubbaloop applications	Scene Understanding (structured JSON), Visual QA (multi-turn sessions), ImageCaptioning

		(self-consistency). Contributed to Kornia.
G5	Benchmark suite	Criterion (Rust) + Python suite: Latency P50/P95/P99, throughput, memory. Published to README. Resolves kornia-rs issue #718.
G6	Kornia-vlm Rust Crate	VLMbackend trait, QwenVLBackend (ort crate), Criterion benches. PR to kornia-rs.
G7	Demo + Documentation	CLI demo (model, backend, image flags), video of live Jetson Inference, Jetson setup guide, full Rustdoc.

9. Week by Week Implementation:

Community Bonding - April 30 to May 25.

PERIOD	OUTPUT
April 30 - May 16	Review of Kornia-vlm, ONNX export docs and setup Jetson env.
May 17 - May 25	Run existing kornia-rs benchmarks and submit technical designs.

PHASE 1: ONNX Export and Foundation (Week 1-4)

WEEK	DATES	DELIVERABLES
1	May 26 - Jun 1	Jetson env ready, CUDA/TRT confirmed. BaseVLM abstract class + VLM config + VLM output with full rust doc. Mock backend unit tests passing on CI. Prototype to mentor.
2	Jun 2 - Jun 8	Qwen2.5-VL-7B AWQ vision encoder exported to ONNX opset 17 with dynamic shapes. Parity Validation: ORT vs Pytorch within 1e-4. ONNX export script with verify flag to prototype.
3	Jun 9 - Jun 15	OrtBackend CPU provider complete, end to end inference test on CI. export_onnx.py verify confirms parity. QwenVL class integrated with ONNX backend option.
4	Jun 16 - Jun 22	CUDA execution provider behind onnx-CUDA feature flag, GPU test passing. Second VLM export start and first pr submit for review.

PHASE 2: Applications and Bubbaloop (Week 5-8)

WEEK	DATES	DELIVERABLES
5	Jun 23 - Jun 29	Scene Understanding application with JSON schema +

		graceful fallback. Robot_navigation mode. COCO validation : recall for object categories. And Midterm PR open.
6	Jun 30 - Jul 6	VisualQA single-shot + multi-turn VQA Session, Benchmarks: Session Q&A latency vs fresh encode, Image Captioning with self consistency.
7	Jul 7 - Jul 13	Bubbaloop VLM pipeline with frame queue and target FPS throttle, Callback routing to Bubbaloop action bus. Integration tests with mock camera feed.
8	Jul 14 - Jul 20	CUDA vs CPU comparative benchmarks committed and Demo video recorded.

PHASE 3: TensorRT, RUST, Benchmarks (Week 9 - 12)

WEEK	DATES	DELIVERABLES
9	Jul 21 - Jul 27	TrtVisionEncoder with ORT tensortRP - engine cached and warm starts. FP16 accuracy validated and kornia-vlm rust crate VLMbackend trait PR opened.
10	Jul 28 - Aug 3	Full benchmark suite: Pytorch vs ONNX vs TRT fp16 vs TRT INT8. Benchmark table published to README
11	Aug 4 - Aug 10	Jetson / Cloud GPU end-to-end tests complete. Final benchmark numbers committed as artifacts. And final demo video recorded - live 2 FPS scene understanding.
12	Aug 11 - Aug 18	All PRs merged or in final review. Full benchmarks, jetson setup guide publishing complete. GSOC final report written and portal submission.

10. Testing And Benchmarking Plan:

1. Unit Tests - All unit tests are designed to run on CPU without requiring a GPU, using mock VLM backends to ensure full CI coverage across environments. The following components are tested:

- BaseVLM interface: Validation of all abstract methods using a mock DummyVLM implementation.
- Throughput metrics: Correct computation of tokens-per-second, including edge cases (zero-latency).
- SceneUnderstanding: Robust JSON parsing with fallback handling for invalid or plain-text outputs.
- LatencyStats: Statistical correctness of mean, P95, and P99 metrics
- OrtBackend: Session creation handling and safe error propagation.

2. Integration Tests - Integration tests validate end-to-end system behavior and performance across different backends:

- End-to-end VLM inference on COCO val2017 sample images with expected output categories.
- ONNX Runtime parity: output comparison against PyTorch within a tolerance of 1e-4 across random samples.
- TensorRT FP16 validation: output quality within 2% of FP32 baseline on VQA benchmarks.
- Jetson stability testing: 1-hour runtime with no out-of-memory (OOM) errors or memory leaks

3. Benchmarking - A Criterion-based Rust benchmark suite + Python benchmark suite covering:

Metric	Tool	Configurations
Latency (ms/image)	Criterion	Mean,P50,P95,P99,min,max,std
Throughput (FPS)	Criterion	Batch 1 and 4, all backends
Memory (MB)	torch.cuda.memory_reserved	Allocated vs reserved, peak VRAM
Warm vs Cold start	Criterion	First inference (engine load) vs cached
Image size scaling	Criterion	224 x 224, 448 x 448, 640 x 480, 1280 x 7

Sample benchmark table format:

```
Model: Qwen2.5-VL-7B-AWQ | Input: 448x448 RGB | Device: Jetson AGX Orin 32GB
──────────────────────────────────────────────────────────────────────────
Backend          Latency (ms)   P95 (ms)   Tok/sec   Peak VRAM (MB)
PyTorch FP16        428           512        18.2        6800
ONNX FP16           398           470        19.6        6500
TRT FP16            332           390        23.5        6200     ← recommended
TRT INT8            288           340        27.1        4800     ← Orin 8 GB
```

11. Why am I the right candidate ?

1. I have hands-on experience with PyTorch, ONNX export, and ONNX Runtime, including debugging export failures, handling dynamic shapes, and optimizing inference. My work in Kornia required deep reasoning about tensor transformations across the pipeline. Most importantly, I have already solved a real ONNX export blocker in Kornia (PR #3463) by making Resize fully compatible with torch.export (dynamo), demonstrating my ability to fix core deployment issues in production code.

2. I can execute independently, I am already in active contact with Edgar Riba and Miquel Farre on Kornia discord. My Rust is at production level for this scope. I have already written and merged Rust code in kornia-rs that touched the ICP pipeline and test infrastructure.
3. I intend to own maintenance of the kornia-vlm inference infrastructure beyond GSOC: updating ONNX/TRT paths as new Jetson SDK versions ship, adding new model integrations (InternVL2, PaliGemma2), and eventually adding a pure RUST LLM decode path with candle or llama.cpp for zero Python development option. I will also write two technical blog posts - one on VLM ONNX export in Rust and one on Jetson VLM deployment - to drive kornia-rs adoption.
4. I have already built a working prototype before submitting this proposal, which significantly reduces execution risk. The remaining work is structured, incremental, and achievable within the GSoC timeline. Also I actively communicate progress, ask for early feedback, and iterate based on maintainer reviews. My prior PRs demonstrate that I respond to feedback quickly and adapt implementations to meet project standards.

Contribution	**Direct Relevance to this Project**
PR #3463 - ONNX Resize Fix	The exact fix required for exporting any Kornia pre-processed VLM pipeline to ONNX. Without this, Stage 1 fails. It is already merged.
PR #722 - ICP geometric metrics	Real production Rust code in kornia-rs. Demonstrates ability to write idiomatic Rust, not just describe plans.
PR #3537 - bbox multi-format	VLMs like Qwen 2.5 VL output bounding boxes in cxcywh format. My fix means VLM grounding outputs flow directly into kornia IoU metrics without a conversion layer.
Issue #718 - benchmarking publishing	Edgar Riba and Sid assigned this to me directly. The GSOC benchmark deliverable extends exactly this work. Mentor trust is already established.

Summary : Three merged PRs targeting exactly the technical problems this project must solve. One assigned issue from the mentor. All contributions in the target organisation. This is not a proposal about what I plan to learn but it is a proposal about what I have already demonstrated.

THANK YOU FOR CONSIDERING THIS PROPOSAL

Arjit Rawat - arjitrawat3@gmail.com - github.com/arjitrawat15